



FGA 0238 - Testes de Software – Turma: 02

Semestre: 2024.1

Nome: João Lucas Pinto Vasconcelos

Matrícula: 190089601

Equipe: 05 – Os Abandonados

Atividade 4 – TDD

1 Funcionalidade

1.1 Identificação da Issue

[145 - Configurar o sistema de log do Django](#)

1.2 Especificação

A issue solicita a configuração do sistema de logging do Django, permitindo a personalização do nível de log e a criação de um diretório específico para armazenar os logs.

1.3 Descrição da Funcionalidade

A funcionalidade a ser implementada envolve a configuração do sistema de logging do Django para que utilize uma variável de ambiente (LOG_LEVEL) para definir o nível de log e crie um diretório (logs) para armazenar os arquivos de log. Isso permitirá que os desenvolvedores ajustem o nível de log conforme necessário, facilitando a depuração e o monitoramento do aplicativo.

1.4 Ciclos

O ciclo de TDD incluirá os seguintes testes:

Teste 1: Verificar se a configuração de logging carrega o nível de log corretamente.

Teste 2: Testar se as mensagens de log são gravadas no console e no arquivo de log.

Teste 3: Garantir que o diretório de logs é criado se não existir.

Teste 4: Verificar se o nível de log pode ser alterado dinamicamente.

Teste 5: Testar se os logs são gravados corretamente em diferentes níveis (DEBUG, INFO, ERROR).

2 Execução

2.1 Primeiro Ciclo

Teste 1: Verificar se a configuração de logging carrega o nível de log corretamente.

```
def test_logging_configuration(django_settings):  
    logging.config.dictConfig(django_settings.LOGGING)  
    logger = logging.getLogger('django')  
    assert logger.level == logging.DEBUG
```

Resultado da execução do teste (falha): O teste falha porque a configuração inicial não está implementada. **Código da funcionalidade:**

```
LOG_LEVEL = os.getenv('LOG_LEVEL', 'DEBUG')
```

Resultado da execução do teste (sucesso): O teste agora passa.

2.2 Segundo Ciclo

Teste 2: Testar se as mensagens de log são gravadas no console e no arquivo de log.

```
def test_logging_output(django_settings, caplog):  
    logging.config.dictConfig(django_settings.LOGGING)  
    logger = logging.getLogger('django')  
    test_message = "Esta é uma mensagem de log de teste."  
    with caplog.at_level(logging.DEBUG):  
        logger.debug(test_message)  
    assert test_message in caplog.text
```

Resultado da execução do teste (falha): O teste falha porque não há implementação para gravar logs.

Código da funcionalidade:

```
LOGGING = {
    'handlers': {
        'console': {
            'level': LOG_LEVEL,
            'class': 'logging.StreamHandler',
        },
        'file': {
            'level': LOG_LEVEL,
            'class': 'logging.FileHandler',
            'filename': LOG_DIR / 'django.log',
        },
    },
}
```

Resultado da execução do teste (sucesso): O teste agora passa.

2.3 Terceiro Ciclo

Teste 3: Garantir que o diretório de logs é criado se não existir.

```
def test_log_directory_exists(django_settings):
    log_dir = django_settings.LOG_DIR
    log_dir.mkdir(parents=True, exist_ok=True)
    assert log_dir.exists()
```

Resultado da execução do teste (falha): O teste falha se o diretório não for criado. **Código da funcionalidade:**

```
LOG_DIR.mkdir(parents=True, exist_ok=True)
```

Resultado da execução do teste (sucesso): O teste agora passa.

2.4 Quarto Ciclo

Teste 4: Verificar se o nível de log pode ser alterado dinamicamente.

```
def test_dynamic_log_level_change(django_settings):  
    logging.config.dictConfig(django_settings.LOGGING)  
    logger = logging.getLogger('django')  
  
    # Altera o nível de log  
    logger.setLevel(logging.WARNING)  
    assert logger.level == logging.WARNING
```

Resultado da execução do teste (falha): O teste falha porque a configuração não permite a alteração dinâmica. **Código da funcionalidade:**

O nível de log é definido pela variável de ambiente, mas pode ser alterado em tempo de execução.

Resultado da execução do teste (sucesso): O teste agora passa após a implementação.

2.5 Quinto Ciclo

Teste 5: Testar se os logs são gravados corretamente em diferentes níveis (DEBUG, INFO, ERROR).

```
def test_logging_levels(django_settings, caplog):
    logging.config.dictConfig(django_settings.LOGGING)
    logger = logging.getLogger('django')

    messages = {
        'debug': "Mensagem de debug",
        'info': "Mensagem de informação",
        'warning': "Mensagem de aviso",
        'error': "Mensagem de erro",
        'critical': "Mensagem crítica",
    }

    for level, message in messages.items():
        with caplog.at_level(getattr(logging, level.upper())):
            getattr(logger, level)(message)
            assert message in caplog.text
```

Resultado da execução do teste (falha): O teste falha se não houver implementação adequada para gravar logs em todos os níveis.

Código da funcionalidade:

A configuração de logging já permite a gravação em diferentes níveis.

Resultado da execução do teste (sucesso): O teste agora passa.

3 Código Fonte Testes

```
import logging
import logging.config
import pytest
from django.conf import settings
from pathlib import Path

@pytest.fixture
def django_settings(settings):
```

```
settings.LOG_LEVEL = 'DEBUG'
settings.LOG_DIR = Path('logs')
settings.LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'verbose': {
            'format': '{levelname} {asctime} {module} {message}',
            'style': '{',
        },
        'simple': {
            'format': '{levelname} {message}',
            'style': '{',
        },
    },
    'handlers': {
        'console': {
            'level': settings.LOG_LEVEL,
            'class': 'logging.StreamHandler',
            'formatter': 'simple',
        },
        'file': {
            'level': settings.LOG_LEVEL,
            'class': 'logging.FileHandler',
            'filename': settings.LOG_DIR / 'django.log',
            'formatter': 'verbose',
        },
    },
    'loggers': {
        'django': {
            'handlers': ['console', 'file'],
            'level': settings.LOG_LEVEL,
            'propagate': True,
        },
    },
}
return settings

def test_logging_configuration(django_settings):
    logging.config.dictConfig(django_settings.LOGGING)

    logger = logging.getLogger('django')

    assert logger.level == logging.DEBUG

    console_handler = next(
```

```
        handler for handler in logger.handlers if isinstance(handler, logging.StreamHandler)
    )
    assert console_handler.level == logging.DEBUG

    file_handler = next(
        handler for handler in logger.handlers if isinstance(handler, logging.FileHandler)
    )
    assert file_handler.level == logging.DEBUG
    assert file_handler.baseFilename.endswith('django.log')

def test_logging_output(django_settings, caplog):
    logging.config.dictConfig(django_settings.LOGGING)

    logger = logging.getLogger('django')

    test_message = "Esta é uma mensagem de log de teste."
    with caplog.at_level(logging.DEBUG):
        logger.debug(test_message)

    assert test_message in caplog.text

    log_file_path = django_settings.LOG_DIR / 'django.log'
    assert log_file_path.exists()

    with open(log_file_path, 'r') as log_file:
        log_contents = log_file.read()
        assert test_message in log_contents

def test_logging_levels(django_settings, caplog):
    logging.config.dictConfig(django_settings.LOGGING)
    logger = logging.getLogger('django')

    messages = {
        'debug': "Mensagem de debug",
        'info': "Mensagem de informação",
        'warning': "Mensagem de aviso",
        'error': "Mensagem de erro",
        'critical': "Mensagem crítica",
    }

    for level, message in messages.items():
        with caplog.at_level(getattr(logging, level.upper())):
            getattr(logger, level)(message)

        assert message in caplog.text
```

```
def test_log_file_creation(django_settings):
    log_file_path = django_settings.LOG_DIR / 'django.log'
    logging.config.dictConfig(django_settings.LOGGING)

    assert log_file_path.exists()

def test_log_directory_exists(django_settings):
    log_dir = django_settings.LOG_DIR
    log_dir.mkdir(parents=True, exist_ok=True) # Cria o diretório se não existir

    assert log_dir.exists()

def test_logger_handlers_removal(django_settings):
    logging.config.dictConfig(django_settings.LOGGING)
    logger = logging.getLogger('django')

    # Remover todos os manipuladores
    logger.handlers.clear()
    assert len(logger.handlers) == 0

def test_logging_format(django_settings, caplog):
    logging.config.dictConfig(django_settings.LOGGING)
    logger = logging.getLogger('django')

    test_message = "Mensagem de teste para formatação."
    with caplog.at_level(logging.DEBUG):
        logger.debug(test_message)

    assert "DEBUG" in caplog.text # Verifica se o nível está presente
    assert "Mensagem de teste para formatação." in caplog.text
```

[test_log.py](#)

4 Resultado Final Execução Testes

index.html

Report generated on 11-Aug-2024 at 12:54:53 by [pytest.html](#) v4.0.2

Environment

Python	3.10.5
Platform	Linux-5.15.153.1-microsoft-standard-WSL2-x86_64-with-glibc2.31
Packages	• pytest: 8.3.2 • pluggy: 1.5.0
Plugins	• cov: 4.1.0 • env: 0.8.1 • django: 4.5.2 • metadata: 3.1.1 • order: 1.1.0 • html: 4.0.2

Summary

8 tests took 00:00:01.

(Un)check the boxes to filter the results.

☒ 0 Failed ☒ 8 Passed ☐ 0 Skipped ☐ 0 Expected failures ☐ 0 Unexpected passes ☐ 0 Errors ☐ 0 Retruns

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Passed	tests/users/test_university_users_endpoint.py::TestUsersEndpoint::test_login_university_user_already_created	207 ms	
Passed	tests/test_log.py::test_logging_configuration	1 ms	
Passed	tests/test_log.py::test_logging_output	1 ms	
Passed	tests/test_log.py::test_logging_levels	2 ms	
Passed	tests/test_log.py::test_log_file_creation	1 ms	
Passed	tests/test_log.py::test_log_directory_exists	0 ms	
Passed	tests/test_log.py::test_logger_handlers_removal	1 ms	
Passed	tests/test_log.py::test_logging_format	1 ms	

5 Código Fonte da Funcionalidade Implementada

```
# Verificar se a variável de ambiente LOG_LEVEL está sendo carregada corretamente
LOG_LEVEL = os.getenv('LOG_LEVEL', 'DEBUG')
# diretório de logs
LOG_DIR = Path('logs')
# Criar o diretório de logs se não existir
LOG_DIR.mkdir(parents=True, exist_ok=True)

LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'verbose': {
            'format': '{levelname} {asctime} {module} {message}',
            'style': '{',
        },
        'simple': {
            'format': '{levelname} {message}',
            'style': '{',
        },
    },
    'handlers': {
        'console': {
            'level': LOG_LEVEL,
            'class': 'logging.StreamHandler',
            'formatter': 'simple',
        },
        'file': {
            'level': LOG_LEVEL,
```

```
        'class': 'logging.FileHandler',
        'filename': LOG_DIR / 'django.log',
        'formatter': 'verbose',
    },
},
'loggers': {
    'django': {
        'handlers': ['console', 'file'],
        'level': LOG_LEVEL,
        'propagate': True,
    },
},
}
```

[settings.py](#)

Comment Close merge request

7 Conclusão

A experiência de desenvolver com TDD foi enriquecedora, pois permitiu que eu focasse nos requisitos da funcionalidade desde o início. A abordagem TDD ajudou a garantir que cada parte do código fosse testada e validada antes de ser integrada, resultando em um sistema de logging mais robusto e confiável. Além disso, a prática de escrever testes antes do código me forçou a pensar cuidadosamente sobre a estrutura e a funcionalidade do que estava sendo implementado, levando a um design mais limpo e modular.